Game of Life finito como máquina de estados finita

Unidade curricular: Lógica Computacional

O *Game of Life* de Conway é um autómato celular. Neste trabalho a grelha é finita: $N \times N$ células **normais** (com $N > 16$), rodeadas por uma moldura de células **iniciais**, de valor aleatório mas constante, e por uma moldura de células **finais**, sempre mortas. As células normais evoluem pela regra B3/S23. Pretende-se construir uma **máquina de estados finita (FSM)** que represente este autómato, com duas relações de transição possíveis (uma só célula muda por transição, ou todas mudam em simultâneo), e verificar duas propriedades:

1. nunca se alcança o estado de erro (todas as células normais mortas);
2. nenhuma célula normal está permanentemente morta ou permanentemente viva.

A abordagem tem duas partes. Primeiro, um **simulador em Python** que executa o autómato e que validamos à mão. Depois, a mesma FSM descrita em **lógica** com o solver **Z3**, que permite perguntar "existe alguma execução que...?" sem enumerar os 2^{N^2} estados. O simulador serve de referência para testar a codificação em Z3.

O notebook segue esta ordem: células e vizinhança, simulador, FSM em Z3, propriedades 1 e 2, testes de escala, conclusão e declaração de uso de LLM.

Escolhas de modelação

Técnica de resolução

Descrevemos a FSM em lógica e usamos o **Z3**, uma das ferramentas sugeridas pela disciplina. O problema é de **verificação** (existe ou não uma execução que viola uma propriedade), e não de otimização, por isso um solver SAT/SMT é a escolha natural. Com $N = 17$ há 2^{289} estados, o que torna impossível enumerá-los. O Z3 responde `sat` e devolve um **modelo** (um contra-exemplo concreto), ou responde `unsat` (não existe tal execução).

Por isso o trabalho tem duas partes. Um **simulador em Python**, validado à mão, e a FSM em Z3, que é comparada com o simulador. Um erro na codificação em Z3 não dá erro nenhum, o solver responderia com confiança sobre o modelo errado, e o simulador serve para o apanhar.

Parâmetros

`N` (dimensão, com $N > 16$) e `MODO` (`"sincrono"` ou `"assincrono"`, a relação de transição) estão numa célula própria. Mudar um deles refaz o notebook.

As células e o estado inicial

A grelha é $(N+2) \times (N+2)$, com índices de 0 a $N+1$, dividida em três classes:

classe	condição	quantidade	comportamento
normais	$1 \le i, j \le N$	N^2	evoluem pela regra B3/S23
iniciais	$i = 0$ ou $j = 0$, com $i, j \le N$	$2N + 1$	constantes, sorteadas com probabilidade $1/2$
finais	$i = N + 1$ ou $j = N + 1$	$2N + 3$	constantemente mortas

Os cantos $(0, N+1)$ e $(N+1, 0)$ contam como **finais** (sempre mortos), para que as contagens do enunciado deem $2N + 1$ e $2N + 3$.

As contagens batem certo: $N^2 + (2N+1) + (2N+3) = (N+2)^2$. Duas células são **vizinhas** se $\max(|\Delta i|, |\Delta j|) = 1$ (as 8 à volta). A moldura existe para que toda a célula normal tenha as suas 8 vizinhas, e os `assert` do notebook verificam estas contagens e esta propriedade.

No estado inicial só as normais com $i, j \le 3$ estão vivas (9 células). As iniciais são sorteadas **uma vez** com uma **semente fixa** (2026) e depois ficam constantes. Os resultados com este sorteio valem só para ele, por isso analisamos também o caso em que as células iniciais podem ter qualquer valor.

Interpretações do enunciado

- **Transição assíncrona:** em cada passo uma célula é escolhida e atualizada pela regra. Se a regra não a manda mudar, o estado repete-se (*stuttering*), em vez de exigir que a célula mude mesmo. Assim todo o estado tem um sucessor. A leitura estrita, em que a célula tem de mudar mesmo, dá os mesmos resultados (ver testes de escala).
- **"Permanentemente":** lido como "a partir de certo instante a célula nunca mais muda", e refutado por um estado estável.
- **Células iniciais:** sorteadas uma vez e constantes. Analisamos também o caso em que podem ter qualquer valor (ver Propriedade 1).

Bibliotecas

- `random` : sorteio das células iniciais.
- `z3` : variáveis, fórmulas e solver.
- `time` : medição dos tempos dos testes de escala.
- `marimo` : o próprio notebook e estes textos.

In [ ]:
```
import marimo as mo
import random
import time
from z3 import Bool, IntVal, BoolVal, Solver, Or, And, Not, Implies,
AtMost, simplify, sat, Sum, If, is_true, unsat
```

In [ ]:
```
# Parâmetros
N = 17
MODO = "sincrono"
assert N > 16
assert MODO in ("sincrono", "assincrono")
```

In [ ]:
```
# Classificação células
normais  = [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
iniciais = [(i, j) for i in range(0, N + 1) for j in range(0, N + 1) if i
== 0 or j == 0]
finais   = [(i, j) for i in range(0, N + 2) for j in range(0, N + 2) if i
== N + 1 or j == N + 1]

assert len(normais)  == N * N
assert len(iniciais) == 2 * N + 1
assert len(finais)   == 2 * N + 3
```

In [ ]:
```
# Células vizinhas
def vizinhas(i, j):
    return [(i+di, j+dj) for di in (-1,0,1) for dj in (-1,0,1) if (di,dj)
!= (0,0)]
```

In [ ]:
```
todas = set(normais) | set(iniciais) | set(finais)
assert len(todas) == (N + 2) ** 2
assert all(v in todas for (i, j) in normais for v in vizinhas(i, j))
```

In [ ]:
```
# Estado inicial das normais
viva_inicialmente = {(i, j): (i <= 3 and j <= 3) for (i, j) in normais}
assert sum(viva_inicialmente.values()) == 9
```

In [ ]:
```
# Sorteio das células iniciais
rng = random.Random(2026)
valor_inicial = {c: rng.random() < 0.5 for c in iniciais}
```

Simulador em Python

O estado é um dicionário `(i, j) → True/False` só com as células normais. O valor de uma célula lê-se assim: se é normal, do estado, se é inicial, do sorteio, e se não é nenhuma das duas, é final e vale `False`.

- `contar_vizinhos(estado, c)` conta os vizinhos vivos de `c`.
- `proxima(viva, n)` é a regra B3/S23: fica viva se tem 3 vizinhos, ou se estava viva e tem 2.
- `passo_sincrono(estado)` calcula todo o estado seguinte **a partir do estado antigo**, sem o alterar durante o cálculo, e é isso que torna a atualização simultânea.
- `desenha(estado)` mostra a grelha: `X` / `–` normal viva/morta, `#` / `.` inicial viva/morta, `~` final.

Testes. Validamos o simulador com contas à mão: os 5 casos da regra, a contagem de vizinhos de células conhecidas ($(1,1)$ tem 3 mais as vizinhas vivas da moldura, $(2,2)$ tem 8, $(10,10)$ tem 0) e o primeiro passo desenhado. Estes resultados são a **referência** para testar o Z3.

In [ ]:
```python
# Valores células
def valor(estado, c):
    if c in estado:
        return estado[c]
    if c in valor_inicial:
        return valor_inicial[c]
    return False


def contar_vizinhos(estado, c):
    return sum(valor(estado, v) for v in vizinhas(*c))

def proxima(viva, n):
    return (n == 3) or (viva and n in (2, 3))
```

In [ ]:
```python
def passo_sincrono(estado):
    return {c: proxima(estado[c], contar_vizinhos(estado, c)) for c in
normais}
```

In [ ]:
```
def desenha(estado, destaque=None):
    viz = set(vizinhas(*destaque)) if destaque else set()
    for i in range(N + 2):
        linha = ""
        for j in range(N + 2):
            c = (i, j)
            if c in estado:      # célula normal
                simbolo = "X" if estado[c] else "–"
            elif c in valor_inicial:        # célula inicial
                simbolo = "#" if valor_inicial[c] else "."
            else:                           # célula final
                simbolo = "~"
            marca = "@" if c == destaque else ("*" if c in viz else " ")
            linha += simbolo + marca
        print(linha)
```

In [ ]:
```
# Testes do simulador
assert [proxima(True, 5), proxima(True, 8), proxima(True, 2),
proxima(False, 3), proxima(False, 2)] \
    == [False, False, True, True, False]

# (1,1): 3 vizinhas normais vivas + as vizinhas da moldura que o sorteio
deixou vivas
_moldura = sum(valor_inicial[v] for v in [(0, 0), (0, 1), (0, 2), (1, 0),
(2, 0)])
assert contar_vizinhos(viva_inicialmente, (1, 1)) == 3 + _moldura
assert contar_vizinhos(viva_inicialmente, (2, 2)) == 8
assert contar_vizinhos(viva_inicialmente, (10, 10)) == 0
print("regra e contagem de vizinhos conferem com as contas à mão")
print("\nPrimeiro passo síncrono:")
desenha(passo_sincrono(viva_inicialmente))
```

```
regra e contagem de vizinhos conferem com as contas à mão

Primeiro passo síncrono:
# . . . # # . . . . . . . . # . # . ~
# – – – – – – – – – – – – – – – – ~
. – – – X – – – – – – – – – – – – ~
. X – X – – – – – – – – – – – – – ~
. X X – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
```

A FSM em lógica (Z3)

Uma FSM é um triplo (S, I, T): estados, estado inicial e relação de transição.

- **Estado:** uma variável booleana por célula normal e por instante t (`c_i_j_t`). A mesma célula em instantes diferentes são variáveis diferentes.
- **Constantes:** as iniciais valem o sorteio e as finais valem `False` , porque nunca mudam.
- **Regra:** `proxima_z3` devolve uma **fórmula** em vez de um valor, que só se resolve quando o solver souber os valores. O número de vizinhos vivos é uma soma de `If(viva, 1, 0)` , porque um booleano do Z3 não é um número.
- **Estado inicial $I(s)$:** cada variável do instante 0 vale o que diz o enunciado.
- **Transição síncrona:** para cada célula c, $p_c = \text{regra}(s, c)$. Só existe um p possível, por isso é determinista (o `passo_sincrono` escrito como restrição).
- **Transição assíncrona:** em cada passo escolhe-se **uma** célula, atualizada pela regra, e as outras ficam iguais. A escolha é feita por um **seletor** booleano por célula e por passo, com a restrição "exatamente um seletor verdadeiro". Escrever "a célula c muda e as outras ficam iguais" para cada c daria uma fórmula quadrática, e com seletores é linear em N^2. Se a regra não manda a escolhida mudar, o estado repete-se, e assim todo o estado tem um sucessor.
- **`trans` :** escolhe a relação pelo valor de `MODO` .

Testes. Cada peça é comparada com o simulador: a regra nos 18 casos possíveis, a contagem de vizinhos nas 289 células, a transição síncrona contra `passo_sincrono` (no estado inicial e em 20 estados aleatórios), e a assíncrona (nunca muda duas células, e só muda as que a regra permita).

In [ ]:
```
# Regra B3/S23 em lógica
def proxima_z3(viva, n):
    return Or(n == 3, And(viva, n == 2))
```

In [ ]:
```
# Teste: regra em lógica = regra em Python
for _viva in (True, False):
    for _n in range(9):
        _res = simplify(proxima_z3(BoolVal(_viva), IntVal(_n)))
        assert bool(_res) == proxima(_viva, _n), (_viva, _n)
print("proxima_z3 coincide com proxima nos 18 casos")
```

```
proxima_z3 coincide com proxima nos 18 casos
```

In [ ]:
```
# Variáveis de estado (uma por célula normal e por instante)
def declara_estado(t):
    return {c: Bool(f"c_{c[0]}_{c[1]}_t{t}") for c in normais}
```

In [ ]:
```
# células iniciais e finais
constante = {c: BoolVal(valor_inicial[c]) for c in iniciais}
```

```
constante.update({c: BoolVal(False) for c in finais})
assert len(constante) == (2 * N + 1) + (2 * N + 3)
```

In [ ]:
```
# Valor de uma célula
def valor_z3(estado, c):
    if c in estado:
        return estado[c]
    return constante[c]
```

In [ ]:
```
# Número de vizinhos vivos (expressão inteira)
def contar_vizinhos_z3(estado, c):
    return Sum([If(valor_z3(estado, v), 1, 0) for v in vizinhas(*c)])
```

In [ ]:
```
# Predicado do estado inicial I(s)
def init_z3(s):
    return And([s[c] == BoolVal(viva_inicialmente[c]) for c in normais])
```

In [ ]:
```
# Transição síncrona T(s, p): todas as células mudam
def regra_z3(s, c):
    return proxima_z3(s[c], contar_vizinhos_z3(s, c))

def trans_sincrona(s, p):
    return And([p[c] == regra_z3(s, c) for c in normais])
```

In [ ]:
```
# Transição assíncrona T(s, p): uma só célula muda
def declara_escolha(t):
    return {c: Bool(f"esc_{c[0]}_{c[1]}_t{t}") for c in normais}

def trans_assincrona(s, p, esc):
    exatamente_uma = And(Or(list(esc.values())), AtMost(*esc.values(), 1))
    por_celula = [
        And(Implies(esc[c], p[c] == regra_z3(s,c)),
           Implies(Not(esc[c]), p[c] == s[c]))
        for c in normais
    ]
    return And(exatamente_uma, And(por_celula))
```

In [ ]:
```
# Escolha do modo
def trans(s, p, t):
    if MODO == "sincrono":
        return trans_sincrona(s, p)
    return trans_assincrona(s, p, declara_escolha(t))
```

In [ ]:
```
# Estado de erro e leitura de um estado a partir de um modelo
def erro_z3(s):
    return And([Not(s[c]) for c in normais])

def le_estado(m, s):
    return {c: is_true(m.eval(s[c], model_completion=True)) for c in
normais}
```

In [ ]:
```
# Testes do Z3 (1): contagem de vizinhos e transição síncrona
_s0, _s1 = declara_estado(0), declara_estado(1)
_sol = Solver()
```

```
_sol.add(init_z3(_s0), trans_sincrona(_s0, _s1))
assert _sol.check() == sat
_m = _sol.model()

for _c in normais:
    _n = _m.eval(contar_vizinhos_z3(_s0, _c),
model_completion=True).as_long()
    assert _n == contar_vizinhos(viva_inicialmente, _c), _c
print("contar_vizinhos_z3 coincide com contar_vizinhos nas", len(normais),
"células")

assert le_estado(_m, _s1) == passo_sincrono(viva_inicialmente)
print("trans_sincrona reproduz passo_sincrono no estado inicial")

# o mesmo, em 20 estados aleatórios
_rng = random.Random(1)
for _ in range(20):
    _est = {c: _rng.random() < 0.5 for c in normais}
    _s, _p = declara_estado(0), declara_estado(1)
    _sol = Solver()
    _sol.add(*[_s[c] == BoolVal(_est[c]) for c in normais],
trans_sincrona(_s, _p))
    assert _sol.check() == sat
    assert le_estado(_sol.model(), _p) == passo_sincrono(_est)
print("trans_sincrona == passo_sincrono em 20 estados aleatórios")
```

```
contar_vizinhos_z3 coincide com contar_vizinhos nas 289 células
trans_sincrona reproduz passo_sincrono no estado inicial
trans_sincrona == passo_sincrono em 20 estados aleatórios
```

In [ ]:

```
# Testes do Z3 (2): transição assíncrona, no estado inicial e em estados
aleatórios
_rng = random.Random(1)
_estados = [viva_inicialmente] + [{c: _rng.random() < 0.5 for c in normais}
for _ in range(5)]

for _est in _estados:
    _s0, _p = declara_estado(0), declara_estado(1)
    _sol = Solver()
    _sol.add(*[_s0[_c] == BoolVal(_est[_c]) for _c in normais],
             trans_assincrona(_s0, _p, declara_escolha(0)))

    # (a) nunca mudam duas ou mais células
    _sol.push()
    _sol.add(Sum([If(_p[_c] != _s0[_c], 1, 0) for _c in normais]) >= 2)
    assert _sol.check() == unsat
    _sol.pop()

    # (b) uma célula pode mudar se e só se a regra manda mudar
    for _c in normais:
        _sol.push()
        _sol.add(_p[_c] != _s0[_c])
        _pode = _sol.check() == sat
        _sol.pop()
        _muda = proxima(_est[_c], contar_vizinhos(_est, _c)) != _est[_c]
        assert _pode == _muda, _c
```

```
print("assíncrona: nunca muda duas células, e só muda as que a regra manda mudar")
print("(verificado no estado inicial e em", len(_estados) - 1, "estados aleatórios)")
```

```
assíncrona: nunca muda duas células, e só muda as que a regra manda mudar
(verificado no estado inicial e em 5 estados aleatórios)
```

Propriedades

Procuramos contra-exemplos com **BMC** (*bounded model checking*): desdobramos a FSM em K transições e pedimos ao Z3 uma sequência $s_0, \ldots, s_K$ em que s_0 é inicial, cada par cumpre T, e s_K é um estado "mau". Testamos $K = 0, 1, 2, \ldots$ por ordem, por isso o primeiro `sat` é o contra-exemplo **mais curto**. Um `unsat` só vale **até** K.

Propriedade 1: nunca se alcança o estado de erro

- **Assíncrono:** o erro é alcançado em **9 transições**, o mínimo possível (há 9 células vivas e cada transição muda uma). A propriedade é **falsa**.
- **Síncrono, com o sorteio usado:** o BMC até $K = 45$ não encontra o erro. O simulador percorre a única trajetória: o ciclo começa em $t = 40$ (um ponto fixo) e o mínimo de células vivas é 4. Como o ciclo começa em $t = 40$, todos os estados alcançáveis aparecem até $K = 40$, por isso o BMC com $K = 45$ já é uma prova, e os dois resultados concordam. Para este sorteio a propriedade é **verdadeira**.
- **Síncrono, qualquer sorteio:** se as células iniciais forem variáveis livres do Z3, o BMC encontra um erro ao fim de **3 transições**, e o simulador confirma-o. A propriedade é **falsa** se o sorteio for parte do problema. É um caso raro: numa amostra de 100 sorteios aleatórios, nenhum levou ao erro.

Propriedade 2: nenhuma célula fica permanentemente morta ou viva

Lemos "permanentemente" como "a partir de certo instante a célula nunca mais muda". Para refutar basta alcançar um **estado estável**, em que a regra não manda mudar nenhuma célula: daí em diante todas ficam constantes. Este contra-exemplo não depende de justiça (*fairness*), porque nele nenhuma célula *pode* mudar.

- **Síncrono:** o ponto fixo da trajetória (5 vivas, 284 mortas) é um estado estável. O Z3 encontra o mesmo estado ao fim de 40 transições, e comparamo-lo com o do simulador.
- **Assíncrono:** o Z3 encontra um estado estável ao fim de 5 transições (4 vivas).

A propriedade é **falsa** nos dois modos.

Com `MODO = "sincrono"` o notebook mostra os tabuleiros do modo síncrono. Os resultados do modo assíncrono (erro em 9 transições, estado estável em 5) vêem-se no

teste de escala, e os tabuleiros dos contra-exemplos vêem-se mudando `MODO` para `"assincrono"`.

Noutra leitura (“nenhuma célula tem o mesmo valor em todos os estados alcançáveis”), no modo síncrono 238 das 289 células nunca estão vivas, por isso a propriedade é também falsa.

In [ ]:
```python
# BMC: procurar o erro até K transições
def bmc(K, mau=erro_z3):
    _solver = Solver()
    _estados = [declara_estado(0)]
    _solver.add(init_z3(_estados[0]))
    for k in range(K + 1):
        if k > 0:
            _estados.append(declara_estado(k))
            _solver.add(trans(_estados[k - 1], _estados[k], k - 1))
        _solver.push()
        _solver.add(mau(_estados[k]))
        if _solver.check() == sat:
            return k, _solver.model(), _estados
        _solver.pop()
    return None
```

In [ ]:
```python
# Propriedade 1: o estado de erro é alcançável?
K1 = 45
res = bmc(K1)

if res is None:
    print(f"Nenhum estado de erro alcançável até {K1} transições")
else:
    _k, _m, _estados = res
    print(f"Erro alcançado ao fim de {_k} transições")
    for _t in range(_k + 1):
        print(f"\n--- t = {_t} ---")
        desenha(le_estado(_m, _estados[_t]))
```

```
Nenhum estado de erro alcançável até 45 transições
```

In [ ]:
```python
# Trajetória síncrona até o estado se repetir
def trajetoria(estado, max_passos=10_000):
    vistos = {}
    vivas = []
    for t in range(max_passos):
        chave = tuple(estado[c] for c in normais)
        if chave in vistos:
            return dict(inicio_ciclo=vistos[chave], comprimento=t -
vistos[chave], vivas=vivas)
        vistos[chave] = t
        vivas.append(sum(estado.values()))
        estado = passo_sincrono(estado)
    return None

traj = trajetoria(viva_inicialmente)
if traj is None:
```

```
    print("Não repetiu nenhum estado em 10 000 passos")
else:
    print("O ciclo começa em t =", traj["inicio_ciclo"], "e tem
comprimento", traj["comprimento"])
    print("Mínimo de células vivas na trajetória:", min(traj["vivas"]))
    print("Alguma vez chegou ao erro?", min(traj["vivas"]) == 0)
```

```
O ciclo começa em t = 40 e tem comprimento 1
Mínimo de células vivas na trajetória: 4
Alguma vez chegou ao erro? False
```

In [ ]:

```
# Propriedade 1 (síncrono) para todas as configurações das células iniciais
_livre = {c: Bool(f"ini_{c[0]}_{c[1]}") for c in iniciais}
_cte = {**_livre, **{c: BoolVal(False) for c in finais}}

def _regra(s, c):
    _n = Sum([If(s[v] if v in s else _cte[v], 1, 0) for v in vizinhas(*c)])
    return proxima_z3(s[c], _n)

def _passo(e, mold):
    return {c: proxima(e[c], sum((e[w] if w in e else mold.get(w, False))
for w in vizinhas(*c)))
            for c in normais}

_sol = Solver()
_est = [{c: Bool(f"l_{c[0]}_{c[1]}_t0") for c in normais}]
_sol.add(*[_est[0][c] == BoolVal(viva_inicialmente[c]) for c in normais])
_k_erro = None
for _k in range(31):
    if _k > 0:
        _est.append({c: Bool(f"l_{c[0]}_{c[1]}_t{_k}") for c in normais})
        _sol.add(And([_est[_k][c] == _regra(_est[_k - 1], c) for c in
normais]))
    _sol.push()
    _sol.add(And([Not(_est[_k][c]) for c in normais]))
    if _sol.check() == sat:
        _k_erro = _k
        break
    _sol.pop()

if _k_erro is None:
    print("Nenhum erro até 30 transições, para qualquer configuração das
células iniciais")
else:
    _m = _sol.model()
    _mold = {c: is_true(_m.eval(_livre[c], model_completion=True)) for c in
iniciais}
    print(f"Com células iniciais livres, o erro é alcançável ao fim de
{_k_erro} transições")
    print("células iniciais vivas nesse contra-exemplo:",
sum(_mold.values()), "de", len(iniciais))

    # confirmação independente, com o simulador
    _e = dict(viva_inicialmente)
    _vivas = [sum(_e.values())]
    for _ in range(_k_erro):
```

```
        _e = _passo(_e, _mold)
        _vivas.append(sum(_e.values()))
    print("simulador, células vivas em cada instante:", _vivas)
    assert _vivas[-1] == 0

    # quão raro é? amostra de sorteios aleatórios
    _R = random.Random(7)
    _n_erros = 0
    for _ in range(100):
        _mold2 = {c: _R.random() < 0.5 for c in iniciais}
        _e = dict(viva_inicialmente)
        _vistos = set()
        for _t in range(400):
            _e = _passo(_e, _mold2)
            if sum(_e.values()) == 0:
                _n_erros += 1
                break
            _ch = tuple(_e[c] for c in normais)
            if _ch in _vistos:
                break
            _vistos.add(_ch)
    print(f"em 100 sorteios aleatórios, {_n_erros} levaram ao erro")
```

```
Com células iniciais livres, o erro é alcançável ao fim de 3 transições
células iniciais vivas nesse contra-exemplo: 13 de 35
simulador, células vivas em cada instante: [9, 3, 1, 0]
em 100 sorteios aleatórios, 0 levaram ao erro
```

In [ ]:

```
# Ponto fixo do modo síncrono
assert traj["comprimento"] == 1, "o ciclo não é um ponto fixo"
ponto_fixo = viva_inicialmente
for _t in range(traj["inicio_ciclo"]):
    ponto_fixo = passo_sincrono(ponto_fixo)

desenha(ponto_fixo)
print("células vivas:", sum(ponto_fixo.values()))
print("é ponto fixo?", passo_sincrono(ponto_fixo) == ponto_fixo)
```

```
# . . . # # . . . . . . . . # . # . ~
# - - - X X - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
. X - - - - - - - - - - - - - - - - ~
# - X - - - - - - - - - - - - - - - ~
# X - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
# - - - - - - - - - - - - - - - - - ~
# - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
# - - - - - - - - - - - - - - - - - ~
# - - - - - - - - - - - - - - - - - ~
. - - - - - - - - - - - - - - - - - ~
~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
células vivas: 5
é ponto fixo? True
```

In [ ]:
```python
# Propriedade 2 (síncrono): células constantes no ponto fixo
vivas_para_sempre = [c for c in normais if ponto_fixo[c]]
mortas_para_sempre = [c for c in normais if not ponto_fixo[c]]
print("permanentemente vivas :", len(vivas_para_sempre), vivas_para_sempre)
print("permanentemente mortas:", len(mortas_para_sempre))
print("Propriedade 2 verdadeira?", not (vivas_para_sempre or
mortas_para_sempre))
```

```
permanentemente vivas : 5 [(1, 4), (1, 5), (4, 1), (5, 2), (6, 1)]
permanentemente mortas: 284
Propriedade 2 verdadeira? False
```

In [ ]:
```python
# Propriedade 2, outra leitura (síncrono): células com o mesmo valor em
todos os estados alcançáveis
_estados = [viva_inicialmente]
for _ in range(traj["inicio_ciclo"]):
    _estados.append(passo_sincrono(_estados[-1]))

_nunca_vivas = [c for c in normais if not any(s[c] for s in _estados)]
_sempre_vivas = [c for c in normais if all(s[c] for s in _estados)]
print("estados alcançáveis:", len(_estados))
print("células que nunca estão vivas  :", len(_nunca_vivas), "de",
len(normais))
print("células que estão sempre vivas :", len(_sempre_vivas))
print("Propriedade 2 verdadeira nesta leitura?", not (_nunca_vivas or
_sempre_vivas))
```

```
estados alcançáveis: 41
células que nunca estão vivas  : 238 de 289
células que estão sempre vivas : 0
Propriedade 2 verdadeira nesta leitura? False
```

In [ ]:
```
# Estado estável: a regra não manda mudar nenhuma célula
def estavel_z3(s):
    return And([regra_z3(s, c) == s[c] for c in normais])
```

In [ ]:
```
# Propriedade 2: procurar um estado estável
K2 = 45
res2 = bmc(K2, estavel_z3)

if res2 is None:
    print(f"Nenhum estado estável alcançável até {K2} transições")
else:
    _k, _m, _estados = res2
    _final = le_estado(_m, _estados[_k])
    print(f"Estado estável alcançado ao fim de {_k} transições")
    print("células vivas nesse estado:", sum(_final.values()))
    desenha(_final)

    # Teste independente com o simulador: a regra não deve mudar nada
    _muda = [c for c in normais if proxima(_final[c],
contar_vizinhos(_final, c)) != _final[c]]
    print("células que a regra ainda mudaria (esperado []):", _muda)

    # No modo síncrono, tem de coincidir com o ponto fixo do simulador
    if MODO == "sincrono":
        print("coincide com o ponto fixo do simulador?", _final ==
ponto_fixo)
```

```
Estado estável alcançado ao fim de 40 transições
células vivas nesse estado: 5
# . . . # # . . . . . . . # . # . ~
# – – – X X – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. X – – – – – – – – – – – – – – – ~
# – X – – – – – – – – – – – – – – ~
# X – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
# – – – – – – – – – – – – – – – – ~
. – – – – – – – – – – – – – – – – ~
~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
células que a regra ainda mudaria (esperado []): []
coincide com o ponto fixo do simulador? True
```

Testes de escala

Medimos o tempo do BMC variando o tamanho da grelha N e a profundidade K, nos dois modos. `medir` repete o modelo para um N qualquer, por isso o primeiro teste confirma que reproduz os resultados do modelo principal. O tempo cresce aproximadamente com N^2 (o número de células) e com K, sem explosão exponencial nos valores testados.

In [ ]:
```
# Escala · Modelo parametrizado em N (só para medir tempos)
def medir(N_, modo, K, procura="erro", semente=2026, estrita=False):
    inicio = time.time()

    normais_ = [(i, j) for i in range(1, N_ + 1) for j in range(1, N_ + 1)]
    iniciais_ = [(i, j) for i in range(0, N_ + 1) for j in range(0, N_ + 1)
if i == 0 or j == 0]
    finais_ = [(i, j) for i in range(0, N_ + 2) for j in range(0, N_ + 2)
if i == N_ + 1 or j == N_ + 1]
    sorteio = random.Random(semente)
    const = {c: BoolVal(sorteio.random() < 0.5) for c in iniciais_}
    const.update({c: BoolVal(False) for c in finais_})

    def estado(t):
        return {c: Bool(f"c_{c[0]}_{c[1]}_t{t}") for c in normais_}

    def regra(s, c):
        n = Sum([If(s[v] if v in s else const[v], 1, 0) for v in
vizinhas(*c)])
        return proxima_z3(s[c], n)

    def trans(s, p, t):
        if modo == "sincrono":
            return And([p[c] == regra(s, c) for c in normais_])
        esc = {c: Bool(f"esc_{c[0]}_{c[1]}_t{t}") for c in normais_}
        return And(Or(list(esc.values())), AtMost(*esc.values(), 1),
                   *[And(Implies(esc[c], And(p[c] == regra(s, c), p[c] !=
s[c]) if estrita else p[c] == regra(s, c)),
                         Implies(Not(esc[c]), p[c] == s[c])) for c in
normais_])

    def mau(s):
        if procura == "erro":
            return And([Not(s[c]) for c in normais_])
        return And([regra(s, c) == s[c] for c in normais_])

    solver = Solver()
    estados = [estado(0)]
    solver.add(And([estados[0][(i, j)] == BoolVal(i <= 3 and j <= 3) for
(i, j) in normais_]))
    for k in range(K + 1):
        if k > 0:
            estados.append(estado(k))
            solver.add(trans(estados[k - 1], estados[k], k - 1))
        solver.push()
        solver.add(mau(estados[k]))
        if solver.check() == sat:
            return k, time.time() - inicio
```

```
        solver.pop()
    return None, time.time() - inicio
```

In [ ]:
```
# Escala · Teste: a versão parametrizada reproduz os resultados do modelo principal
_conhecidos = [
    ("assincrono", 12, "erro", 9),
    ("assincrono", 12, "estavel", 5),
    ("sincrono", 20, "erro", None),
    ("sincrono", 45, "estavel", 40),
]
for _modo, _K, _procura, _esperado in _conhecidos:
    _k, _t = medir(17, _modo, _K, _procura)
    print(f"{_modo:10} {_procura:7} -> {_k}  (esperado {_esperado})  {_t:.1f}s")
    assert _k == _esperado, (_modo, _procura)
print("A versão parametrizada reproduz os resultados do modelo principal")
```

```
assincrono erro    -> 9  (esperado 9)  2.4s
assincrono estavel -> 5  (esperado 5)  2.4s
sincrono   erro    -> None  (esperado None)  2.3s
sincrono   estavel -> 40  (esperado 40)  8.8s
A versão parametrizada reproduz os resultados do modelo principal
```

In [ ]:
```
# Leitura estrita do modo assíncrono: a célula escolhida tem de mudar mesmo
for _procura, _esperado in (("erro", 9), ("estavel", 5)):
    _k, _t = medir(17, "assincrono", 12, _procura, estrita=True)
    print(f"leitura estrita, {_procura:7} -> {_k}  (leitura com stuttering: {_esperado})  {_t:.1f}s")
    assert _k == _esperado
print("As duas leituras do modo assíncrono dão os mesmos resultados")
```

```
leitura estrita, erro    -> 9  (leitura com stuttering: 9)  1.7s
leitura estrita, estavel -> 5  (leitura com stuttering: 5)  1.6s
As duas leituras do modo assíncrono dão os mesmos resultados
```

In [ ]:
```
# Escala · Tempo do BMC em função de N
print("   N | variáveis/passo | síncrono (K=20) | assíncrono (K=12)")
for _n in (17, 20, 25, 30, 40):
    _ks, _ts = medir(_n, "sincrono", 20)
    _ka, _ta = medir(_n, "assincrono", 12)
    print(f"{_n:>4} | {_n * _n:>15} | {_ts:>12.1f} s  | {_ta:>11.1f} s  (erro em {_ka})")
```

```
  N | variáveis/passo | síncrono (K=20) | assíncrono (K=12)
 17 |             289 |          2.3 s  |          2.7 s  (erro em 9)
 20 |             400 |          3.2 s  |          3.4 s  (erro em 9)
 25 |             625 |          4.9 s  |          6.3 s  (erro em 9)
 30 |             900 |          7.1 s  |          8.4 s  (erro em 9)
 40 |            1600 |         12.7 s  |         17.6 s  (erro em 9)
```

In [ ]:
```
# Escala · Tempo do BMC em função da profundidade K (N = 17)
print("modo        |   K | tempo | resultado")
for _modo, _ks in (("sincrono", (5, 10, 20, 40, 60)), ("assincrono", (3, 6, 9))):
    for _K in _ks:
        _k, _t = medir(17, _modo, _K)
```

```
        print(f"{_modo:11} | {_K:>3} | {_t:>4.1f}s | {'erro em ' + str(_k)
if _k is not None else 'sem erro'}")
```

```
modo        |   K | tempo | resultado
sincrono    |   5 |  0.7s | sem erro
sincrono    |  10 |  1.2s | sem erro
sincrono    |  20 |  2.4s | sem erro
sincrono    |  40 |  4.8s | sem erro
sincrono    |  60 |  7.0s | sem erro
assincrono  |   3 |  0.5s | sem erro
assincrono  |   6 |  1.1s | sem erro
assincrono  |   9 |  2.6s | erro em 9
```

Conclusão

Modelámos o *Game of Life* finito como uma **máquina de estados finita** com as duas relações de transição do enunciado, escolhidas pelo parâmetro `MODO`, e verificámos as duas propriedades com o **Z3**, usando um simulador em Python como referência.

Resultados

	`"sincrono"`	`"assincrono"`
Propriedade 1 (nunca se alcança o erro)	**falsa**: há configurações das células iniciais que levam ao erro em 3 transições. Para o sorteio usado (semente 2026) é verdadeira (ponto fixo em $t = 40$, com pelo menos 4 vivas)"	**falsa**: o erro é alcançado em **9 transições**, o mínimo possível
Propriedade 2 (nenhuma célula permanentemente morta ou viva)	**falsa**: o ponto fixo deixa as 289 células constantes (5 vivas, 284 mortas)	**falsa**: há um estado estável alcançável em 5 transições (4 vivas)

Decisões de modelação e o que aprendemos

1. **Simulador como referência.** Um erro na codificação em Z3 não dá erro nenhum, por isso cada peça em lógica foi comparada com o simulador: a regra nos 18 casos, a contagem de vizinhos nas 289 células, a transição síncrona contra `passo_sincrono`, e a assíncrona (nunca muda duas células, e só muda as que a regra manda mudar). O estado estável encontrado pelo Z3 no modo síncrono coincide com o ponto fixo do simulador, por dois caminhos independentes.
2. **Seletores no modo assíncrono.** Um booleano por célula e por passo, com "exatamente um verdadeiro", dá uma fórmula linear em N^2, em vez de quadrática.
3. **BMC e os seus limites.** O BMC dá o contra-exemplo mais curto, mas um `unsat` só vale até K. No modo síncrono fechámos a propriedade 1, para o sorteio usado, porque o sistema é determinista e finito: a trajetória repete um estado ao fim de 40 passos, e o BMC com $K = 45$ concorda.

4. **"Permanentemente".** Lemos como "a partir de certo instante a célula nunca mais muda", e refutámos com um estado estável, que não depende de justiça (*fairness*).

Escala

O tempo do BMC cresce de forma aproximadamente proporcional ao número de células (N^2) e à profundidade K, sem explosão exponencial nos valores testados. Na máquina do autor, com $K = 20$ no modo síncrono, passou de cerca de 2 s para $N = 17$ a cerca de 13 s para $N = 40$ (o modo assíncrono, com $K = 12$, de cerca de 2 s a 17 s). Para $N = 17$ e o modo síncrono, o tempo foi de 0,7 s para $K = 5$ a 6,8 s para $K = 60$. Os tempos dependem da máquina, e cada ponto é uma só execução.

Limitações

- As células iniciais foram sorteadas **uma vez** (semente 2026), e a propriedade 1 síncrona só é verdadeira para esse sorteio: com células iniciais livres, o erro é alcançável em 3 transições. Não calculámos que fração das configurações o provoca (numa amostra de 100 sorteios, nenhuma).
- A propriedade 2 foi lida também como "nenhuma célula tem o mesmo valor em todos os estados alcançáveis". No modo síncrono é falsa nesta leitura, mas no assíncrono não a analisámos: exigiria provar que um estado nunca é alcançável, e o BMC só garante até K.
- Os testes de escala medem o custo de verificar até um K fixo, e não de provar uma propriedade em geral.

Declaração de Uso de LLM

Em conformidade com as diretrizes da disciplina, declara-se a utilização do assistente LLM Claude como ferramenta de apoio ao desenvolvimento e discussão de alternativas de modelação.

- **Link do Diálogo:** https://claude.ai/share/2983ce9c-ba6d-4d1b-8ca7-4f723d7b68fd
- **Auxílios:**
  - Explicação do enunciado e da interpretação das contagens das células iniciais e finais.
  - Sugestão da técnica de resolução (Z3, *bounded model checking*) e da estrutura do notebook.
  - Geração dos testes de correção e dos testes de escala.
  - Ajuda na depuração de erros do marimo e do código.
  - Redação dos textos explicativos do notebook.